

课程笔记

CS336 Lecture 17: Policy Gradient Mechanics and GRPO

基于课程字幕与讲义脚本整理

2025 年 3 月 26 日



视频作者/频道: CS336 / Stanford

发布日期: 2025-03-26

视频时长: 约 31 分钟

视频链接: 未填写

目录

1 引言：为什么要讲 policy gradient	2
2 语言模型里的 RL 设定	2
2.1 state, action, reward	2
2.2 目标函数	3
3 Policy Gradient 的基本推导	3
3.1 从期望奖励到 log-derivative trick	3
3.2 naive policy gradient	3
4 为什么要 baseline	4
4.1 高方差问题	4
4.2 baseline 不改变目标	4
4.3 方差为什么会下降	4
4.4 advantage function	4
5 GRPO 的直觉	5
5.1 什么是 group relative	5
5.2 和 PPO 的关系	5
6 一个可训练的 toy problem	5
6.1 排序任务	5
6.2 模型结构	6
6.3 采样和打分	6
7 delta 的几种选择	6
8 loss 的几种形式	7
8.1 naive loss	7
8.2 old policy ratio	7
8.3 clipping	7
8.4 KL penalty	7
9 训练流程	8
9.1 GRPO 风格的循环	8
9.2 实验现象	8
10 总结	9
10.1 这一讲的结论	9
10.2 最后的提醒	9
10.3 参考	9

1 引言：为什么要讲 policy gradient

这一讲是本课程的最后一讲，由于前一讲已经概览了 *reinforcement learning from verifiable rewards* 和策略梯度类方法，这一讲不再引入全新框架，而是把 policy gradient 和 GRPO 的机制掰开揉碎，重点看三个问题：

1. 在语言模型里，RL 的 **state**、**action**、**reward** 到底对应什么；
2. 为什么 naive policy gradient 会有高方差，baseline 和 advantage 为什么能缓解它；
3. GRPO 在代码里到底怎么跑，为什么要用 group 内平均奖励做比较。

这一讲的核心

- 如果你能度量，就能优化，但 RL 的难点在于度量很稀疏、很噪声。
- 策略梯度本身不神秘，本质上就是对 $\log \pi(a | s)$ 乘上一个奖励信号。
- baseline 的作用是降方差，不是改变优化目标。

2 语言模型里的 RL 设定

2.1 state, action, reward

把整段 response 记作一个动作 a ，语言模型 RL 的对应关系非常直接：

- **State** s : prompt 加上已经生成的 response 前缀。
- **Action** a : 生成下一个 token，或者在 outcome reward 视角里，直接把整段 response 看成一个动作序列。
- **Reward** $R(s, a)$: 整段 response 的好坏，本文只讨论 **outcome reward** 和 **verifiable reward**。

字幕里强调了一点：在语言模型场景中，transition dynamics 基本就是拼接操作。你做出的下一状态就是

$$s' = s + a$$

也就是把 token 继续 append 到上下文里。这和机器人控制非常不同，后者的状态由物理世界决定，语言模型的状态则更像是“自己写出来的中间草稿”。

语言模型里的 state 很“虚”

在 robotics 中，state 可能是关节角、速度、图像；在 LM 中，state 只是 token 序列。这个差别意味着：

- LM 可以自由构造 scratchpad；
- LM 可以在 test time 做 planning / search；
- 但 reward 往往更稀疏，正确与错误的边界也更硬。

2.2 目标函数

如果把 prompt 视作由环境给出的 s , response 视作 policy 采样出来的 a , 那么目标就是最大化期望奖励:

$$\max_{\pi} \mathbb{E}_{s \sim p(s), a \sim \pi(\cdot|s)} [R(s, a)].$$

在 outcome reward 设定下, reward 只在整段输出结束后给出, 因此 discounting、bootstrapping 这些经典 RL 概念在这里不那么核心。

3 Policy Gradient 的基本推导

3.1 从期望奖励到 log-derivative trick

为了记号简洁, 把整段 response 当作一个整体动作 a 。那么

$$\mathbb{E}[R] = \int p(s) \pi(a | s) R(s, a) ds da.$$

对 policy 参数求梯度:

$$\nabla \mathbb{E}[R] = \int p(s) \nabla \pi(a | s) R(s, a) ds da = \int p(s) \pi(a | s) \nabla \log \pi(a | s) R(s, a) ds da.$$

于是得到熟悉的形式:

$$\nabla \mathbb{E}[R] = \mathbb{E}[\nabla \log \pi(a | s) R(s, a)].$$

policy gradient 的直觉

- 采到好样本, 就提高它的概率;
- 采到差样本, 就降低它的概率;
- 变化的幅度由 reward 决定。

3.2 naive policy gradient

最朴素的做法就是:

1. 采样一个 prompt s ;
2. 从当前 policy 采样 response a ;
3. 用 $\nabla \log \pi(a | s) R(s, a)$ 更新参数。

这和 supervised fine-tuning 很像, 只不过监督数据不是人工标注, 而是模型自己采样出来后再按 reward 过滤和加权。

稀疏奖励会把更新打空

如果 $R \in \{0, 1\}$, 多数样本 reward 都是 0, 那么很多 batch 的梯度就是近似 0。模型如果一开始就很差, 可能连“碰到正奖励样本”的机会都很少。

4 为什么要 baseline

4.1 高方差问题

在语言模型 RL 中，reward 往往是稀疏的、延迟的、并且分布差异很大。字幕里的 toy 例子把这个问题讲得很直白：

- state s_1 的 reward 可能是 11 或 9；
- state s_2 的 reward 可能是 0 或 2；
- 直接拿 reward 做更新，会出现“9 比 2 大，所以看起来更值得强化”的局部错觉。

但是从全局看， s_2 明显更难、reward 更低，不能只看绝对值。这里真正要比较的是“**这个 action** 比在当前 state 下的平均水平好多少”。

4.2 baseline 不改变目标

引入 baseline $b(s)$ 后，我们优化的是

$$\mathbb{E}[R - b(s)].$$

只要 baseline 不依赖动作 a ，这个变换不会改变原来的最优解，因为

$$\mathbb{E}[b(s)] = \int p(s) b(s) ds$$

与 policy 无关，因而只是加了一个常数项。

baseline 的要求

baseline 可以依赖 state，不能依赖 action。直观上，它是“当前 state 下你本来就该得到多少”的参照线。

4.3 方差为什么会下降

在 toy 例子中，原始 reward 的方差大约是 5.3；如果对两个 state 分别取 baseline 10 和 1，方差会降到约 1.1。这个例子想说明的不是某个数值，而是一个原则：

baseline 的作用

- 保留期望优化方向；
- 压缩无用的绝对量级差异；
- 让梯度更新更稳定、更快收敛。

4.4 advantage function

经典 RL 里有：

$$V(s) = \mathbb{E}[R \mid s], \quad Q(s, a) = \mathbb{E}[R \mid s, a],$$

以及 advantage:

$$A(s, a) = Q(s, a) - V(s).$$

在这一讲的 outcome reward 设定里, Q 和 R 基本可以视为同一个量, 因此当 baseline 取成

$$b(s) = \mathbb{E}[R \mid s]$$

时, $R - b(s)$ 就是 advantage 的形式。

难点不在公式, 而在估计

理论上最优 baseline 可以写出 closed form, 但实际里很难精确计算期待值, 所以大家通常用样本平均、group mean, 或者 value model 去近似它。

5 GRPO 的直觉

5.1 为什么是 group relative

GRPO 的关键不在于名字, 而在于它利用了语言模型的自然分组结构:

- 同一个 prompt 可以采样出多条 response;
- 这些 response 天然构成一个 group;
- group 内平均 reward 就是一个很自然的 baseline。

这就是“relative”的来源。它比较的是“同一 prompt 下, 谁比同组其他 response 更好”。

为什么 GRPO 适合语言模型

对同一个 prompt 采样多条 response, 本来就是 LM 训练里最自然的采样单位, 因此 baseline 不必再从全局状态空间拟合, 而可以在 group 内做相对比较。

5.2 和 PPO 的关系

课程里把 GRPO 描述成一种 **不需要 critic** 的 PPO 风格简化版。你仍然会看到:

- policy ratio;
- clipping;
- 参考策略或旧策略;
- KL regularization。

但 group 结构让 baseline 近似得更自然, 所以实现上可以更轻。

6 一个可训练的 toy problem

6.1 排序任务

为了能在本地跑通, 讲义脚本构造了一个很小的 RL 任务: 给定一组数字, 模型要输出排序后的序列。

- Prompt: 一组数字;
- Response: 希望是排序后的数字;
- Reward: 衡量 response 和排序答案接近的程度。

这里故意不直接用“完全正确 = 1, 否则 0”的极端奖励, 而是设计了两个版本:

1. **位置匹配奖励**: response 与 ground truth 在多少位置上一致;
2. **inclusion + ordering 奖励**: 既看 token 是否出现, 也看局部顺序是否合理。

为什么要给 partial credit

如果只给 0/1 奖励, 大量样本都没有学习信号。给 partial credit 可以让模型在“还没完全做对”时也能得到有用反馈。

6.2 模型结构

脚本里的 toy model 很小, 但足够展示 policy gradient 的机制:

- prompt 和 response 长度固定;
- 每个位置有独立的 encode/decode 参数;
- 最后通过 embedding weight 生成 logits。

这不是一个真实的 transformer, 而是一个便于分析的最简模型。

6.3 采样和打分

训练流程大致如下:

1. 输入 prompt;
2. 从当前模型采样多个 responses;
3. 用 reward function 对每条 response 打分;
4. 根据 delta 形式构造更新信号;
5. 用 policy gradient 更新模型。

7 delta 的几种选择

脚本里把用于更新的量统一记成 δ , 然后尝试了几种常见做法:

- **rewards**: 直接用 raw rewards;
- **centered rewards**: 减去 group mean;
- **normalized rewards**: 再除以标准差;
- **max rewards**: 只保留组内最大 reward。

最常见的是 centered rewards

在同一个 prompt 的多个 response 里，减掉 group mean 是最自然的 baseline 近似。它对应“相对同组平均表现的好坏”。

8 loss 的几种形式

8.1 naive loss

最直接的 loss 是

$$\mathcal{L}_{\text{naive}} = -\mathbb{E}[\log \pi(a | s) \delta].$$

它就是把 log probability 和 reward 信号直接乘起来。

8.2 old policy ratio

如果引入旧策略 π_{old} ，则会出现 ratio：

$$\frac{\pi(a | s)}{\pi_{\text{old}}(a | s)}.$$

脚本里把它写成 log prob 差：

$$\exp(\log \pi - \log \pi_{\text{old}}).$$

旧策略必须 freeze

‘old policy’ 只能当常量用，不能让梯度穿过去。否则 ratio 的意义会被破坏，整个优化会偏掉。

8.3 clipping

和 PPO 类似，ratio 太大或太小时都不稳定，所以会做 clipping：

$$r = \frac{\pi}{\pi_{\text{old}}}, \quad \tilde{r} = \text{clip}(r, 1 - \epsilon, 1 + \epsilon).$$

最后用 unclipped 和 clipped 的较小者构成保守更新。

8.4 KL penalty

如果担心 RL 把模型带偏太远，还可以加 reference model 的 KL penalty：

$$\text{KL}(p||q) = \mathbb{E}_{x \sim p} \left[\log \frac{p(x)}{q(x)} \right].$$

脚本里用了一个数值上可算的估计：

$$\mathbb{E}_p \left[\frac{q}{p} - \log \frac{q}{p} - 1 \right].$$

KL penalty 的作用

它是“别忘了原来的模型能力”的安全带。你想让模型获得新能力，但又不想把旧能力完全冲掉。

9 训练流程

9.1 GRPO 风格的循环

脚本中的训练循环可以概括为：

1. 采样一批 prompts；
2. 对每个 prompt 采样多个 responses；
3. 计算 rewards 和 deltas；
4. 可选地计算 reference model 的 log probs；
5. 用当前 policy 的 log probs 算 loss；
6. 反向传播，更新参数。

9.2 实验现象

课程脚本尝试了三种设置：

- raw rewards；
- centered rewards；
- normalized rewards。

结论是：

- raw rewards 往往学得慢，容易卡住；
- centered rewards 更稳定一些；
- normalized rewards 不一定有明显增益，甚至可能引入不必要的偏差；
- 这类 toy RL 任务本身就很容易卡在局部最优。

RL 不是“设个 reward 就会自动学会”

即便任务和 reward 都写得很清楚，policy gradient 仍然可能因为方差、初始化和超参问题而学不动。

10 总结

10.1 这一讲的结论

四个核心 takeaway

1. 在 LM 里，RL 的 state/action/reward 定义很自然，但 reward 往往非常稀疏。
2. Naive policy gradient 的形式简单，但方差很大。
3. Baseline 和 advantage 的本质是降方差，不是改目标。
4. GRPO 利用了同一 prompt 下多个 response 的 group 结构，所以在 LM 场景里特别合适。

10.2 最后的提醒

这一讲其实在传达一个很朴素但重要的判断：

如果你能度量某个东西，就可以把它塞进 optimization loop 里，但真正困难的是让这个度量足够稳定、足够密、足够有用。

这也是为什么 policy gradient 在语言模型里既强大又麻烦：它给了你直接优化目标的入口，也把训练稳定性问题完整暴露了出来。

10.3 参考

- Schulman et al., Proximal Policy Optimization Algorithms
- Deep RL textbook / CS224R policy gradient notes
- GRPO / verifiable rewards 相关课程材料